Software Testing

Sample study notes written for StudyBuddy. Free to share.

Why we test

Testing is the process of running software to find defects and to gain confidence that it behaves as intended. A test cannot prove that a program has no bugs; it can only show the presence of bugs for the inputs it tries. Good testing therefore chooses inputs carefully, so that a small number of tests covers as many kinds of behaviour as possible.

Three terms are often confused. An error is a human mistake, such as misunderstanding a requirement. A fault, or defect, is the result of that mistake in the code. A failure is what a user observes when the faulty code runs and the program behaves incorrectly. A fault that is never executed causes no failure, which is why untested code can hide defects for a long time.

Testing also has a second purpose: automated tests act as a safety net during change. When a developer refactors code, a passing test suite gives evidence that the behaviour did not change. This regression testing, rerunning existing tests after every change, is one of the main reasons teams automate their tests.

Levels of testing

Unit testing checks one small piece of code, usually a single class or method, in isolation from the rest of the system. Unit tests are fast, run hundreds of times a day and point directly at the broken code when they fail.

Integration testing checks that several units work together correctly, for example a service class together with a real database. Integration tests find problems in the connections between units, such as mismatched assumptions about data formats.

System testing checks the complete application against its requirements, as a user would use it. Acceptance testing is done with or by the customer to decide whether the system is ready to be delivered.

The test pyramid summarises a common balance: many fast unit tests at the base, fewer integration tests in the middle and only a small number of slow end-to-end system tests at the top.

Black-box test design

Black-box testing designs tests from the specification alone, without looking at the code.

Equivalence partitioning divides the possible inputs into groups, called partitions, that the program should treat the same way, and picks one representative from each. For a method that accepts an exam mark from 0 to 100 and returns pass for 50 or more, the partitions are: below 0 (invalid), 0 to 49 (fail), 50 to 100 (pass) and above 100 (invalid). Four tests, one per partition, cover all the kinds of behaviour.

Boundary value analysis adds tests at the edges of each partition, because off-by-one mistakes are among the most common defects. For the exam example the boundary values are -1, 0, 49, 50, 100 and 101. A developer who wrote "mark > 50" instead of "mark >= 50" is caught by the test with 50.

White-box test design and coverage

White-box testing designs tests from the structure of the code. Coverage measures how much of the code a test suite executes.

Statement coverage is the percentage of statements executed at least once. Branch coverage is the percentage of decision outcomes, the true and false sides of every if and loop condition, that were taken. Branch coverage is stronger: a single test can execute every statement of an if without an else while never taking the false branch.

High coverage shows which code was run, not that it was checked. A test with no assertions can reach 100% statement coverage and still find nothing. Coverage is best used to find untested code, not as a goal in itself.

Writing good unit tests

A common structure is Arrange, Act, Assert. Arrange creates the objects and data the test needs, Act calls the method being tested, and Assert checks the result. Each test should check one behaviour and have a name that describes it, such as rejectsMarksAbove100.

Good unit tests are fast, independent of each other and of the order in which they run, repeatable on any machine and self-checking, so that they pass or fail without a person reading the output. A test that sometimes passes and sometimes fails without any change to the code is called flaky; flaky tests are harmful because developers learn to ignore failures.

In Java, JUnit is the standard framework. Test methods are marked with @Test, and assertions such as assertEquals(expected, actual) and assertThrows(...) check results.

Test doubles

A unit often depends on other objects that are slow, unpredictable or not yet written, such as a database, a web service or the system clock. A test double replaces such a dependency during a test.

A stub returns fixed answers that the test prepares in advance. A fake is a simple working implementation, such as an in-memory list used instead of a real database. A mock records how it was called, so the test can verify that the unit made the expected calls. A spy wraps a real object and records the calls made to it.

Test doubles only work if the unit lets its dependencies be replaced. This is why code that depends on interfaces, and receives its dependencies through its constructor as the dependency inversion principle recommends, is much easier to test than code that creates its

own database connection or calls a static method directly.

Test-driven development

Test-driven development, or TDD, reverses the usual order: the test is written before the code. Work proceeds in short cycles of three steps. Red: write a small test for the next piece of behaviour and watch it fail. Green: write the simplest code that makes the test pass. Refactor: improve the design of the code and the tests while keeping all tests passing.

TDD produces a thorough test suite as a side effect, keeps each step small and pushes the design towards small, loosely coupled units, because code that is hard to test is noticed immediately. Its costs are the discipline it needs and the time to maintain the tests when requirements change.

Summary

Idea	Key point
Error, fault, failure	Mistake, defect in code, observed wrong behaviour
Test levels	Unit, integration, system, acceptance
Equivalence partitioning	One test per group of equivalent inputs
Boundary value analysis	Test the edges of each partition
Branch coverage	Every true and false outcome taken
Test doubles	Stub, fake, mock, spy
TDD	Red, green, refactor